

TP – moteur de jeu 2D



Où l'on s'affiche, sans paraître décalé. Où l'on bouge, mais uniquement contrôlé. Où l'on accepte le cadre, par simplicité. Où l'on installe des ennemis, à qui l'on donne des valeurs, nourries par notre intelligence. Et où, dans le fond, il ne nous reste qu'une liberté : réécrire les règles du monde, pour sortir du cadre dans lequel nous nous sommes nous-mêmes enfermés.

1 Première version

🕒 de 3 à 5 h

Objectifs

- Mettre en place les outils de développement et la bibliothèque graphique choisie.
- Afficher correctement le joueur à l'écran.
- Implémenter une première version des déplacements du joueur.

💡 Moteur de jeu

Un **moteur de jeu** est un logiciel chargé de fournir les mécanismes de base pour un jeu vidéo : gérer le temps, les déplacements, les collisions, les événements clavier ou souris, etc.

Un tel moteur ne s'occupe en général ni de l'affichage des éléments à l'écran, ni des règles propres au jeu.

Dans un premier temps, nous allons mettre en place l'environnement de développement pour notre moteur de jeu. Les responsabilités seront séparées :

1. La **bibliothèque graphique** fournit les outils permettant d'afficher des éléments à l'écran, de récupérer des événements clavier ou souris, etc.
2. Le **moteur de jeu** constitue le coeur du TP. Il fait évoluer le monde simulé et gère les interactions entre ses composants.
3. Le **jeu** utilise le moteur de jeu et la bibliothèque graphique pour implémenter ses propres règles (scénarios, objectifs, changement de scènes, etc.).

Pour ce TP, vous pouvez **ne pas respecter** cette séparation dans la mesure où l'objectif principal reste les mathématiques qui se logent dans un moteur de jeu. En revanche, si vous souhaitez coder proprement en séparant les responsabilités, vous pouvez le faire !

Contenu

Moteur de jeu 2D

1	Première version . .	1
2	Méthode d'Euler . .	4
3	Déplacements	9
4	Forces	12
5	Géométrie du joueur	13
6	Autres entités	15
7	Pour aller plus loin .	21

Version 1 – juillet 2026

Coefficient de difficulté : 0,6

Le chapô est librement inspiré de la plume de Marc-André Selosse.

1.1 Choix de la bibliothèque graphique

Dans cette première partie, vous devez choisir une bibliothèque graphique pour pouvoir tester votre moteur, puis l'installer sur votre poste de travail.

Ce choix est libre. Voici quelques recommandations que vous pouvez utiliser :

	Cible	Difficulté	Recommandée pour
Win32	Windows	★★	comprendre un moteur de jeu
Qt	multi	★★★	coder plus proprement un moteur de jeu
SFML	multi	★★★★	comprendre, coder et réutiliser votre moteur de jeu

Pour chaque bibliothèque mentionnée ci-dessus, un squelette minimal est disponible (projet CMake).

1.2 Coordonnées physiques vs écran

Du fait du partage des responsabilités entre le moteur de jeu et la bibliothèque graphique, notre moteur ne peut pas raisonner en termes de positions avec des pixels. C'est une notion liée à l'écran. Le moteur de jeu travaillera exclusivement dans le monde physique. Les coordonnées écran ne seront utilisées qu'au moment de l'affichage.

On propose donc ici de bien distinguer deux types de coordonnées :

- Les **coordonnées physiques**, connues du moteur de jeu et qui représenteront les positions dans le monde physique simulé. On pourra prendre par exemple comme unité le mètre.
- Les **coordonnées écran**, données à la bibliothèque graphique, pour afficher les composants sur l'écran. L'unité est le pixel.

Pour passer d'un monde à l'autre, des outils de conversion sont nécessaires.

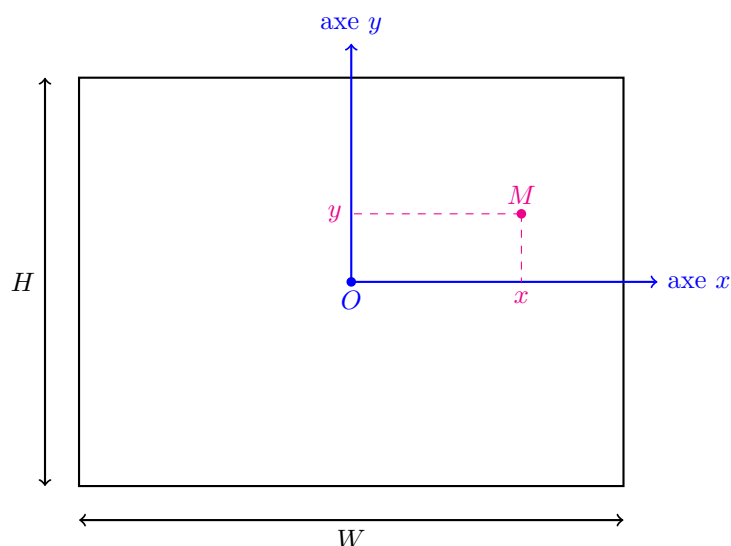


Figure 1. Ecran représenté en noir, de largeur W et de hauteur H (en pixels). Repère physique Oxy représenté en bleu. Exemple d'un point M sur l'écran, et de ses coordonnées physiques x et y .

Remarque

La difficulté indiquée dans le tableau tient compte (1) de l'effort pour inclure la bibliothèque dans votre projet et (2) de l'effort à l'utilisation.

Pour vous aider dans votre choix :

- Win32 est immédiat à configurer, mais l'API est ancienne.
- Qt est relativement simple à configurer, et à utiliser, mais lourd à déployer.
- SFML est plus délicat à installer, sans doute plus nouveau pour vous, mais beaucoup plus adapté aux jeux vidéos.

Remarque

Dans ce document, les coordonnées du monde physique seront notées en minuscules (par exemple : x , d), tandis que les coordonnées du monde écran seront notées en majuscules (par exemple : X , W).

(1) On note H la hauteur de l'écran en pixels et W sa largeur. Le monde physique sera centré sur l'écran. Quelle est la position écran du centre du monde ?

(2) On note z le facteur de zoom, qu'on exprime en pixels par mètre. Pour une distance d dans le monde physique, que vaut la distance D à l'écran ?

(3) Pour un point M de coordonnée horizontale x dans le monde physique, que vaut sa coordonnée X dans le monde écran ?

(4) Inversement, connaissant X , comment retrouver x ?

(5) Pour un point M de coordonnée verticale y dans le monde physique, que vaut sa coordonnée Y dans le monde écran ? Attention : en général les bibliothèques graphiques comptent les distances verticales à partir du haut de la fenêtre (l'axe des Y est donc inversé!).

(6) Inversement, connaissant Y , comment retrouver y ?

(7) A partir des questions précédentes, proposez l'implémentation de la fonction suivante qui convertit des coordonnées physiques en coordonnées écran :

```
using ScreenPoint = std::pair<int, int>;
using WorldPoint = std::pair<double, double>;

ScreenPoint toScreen(const WorldPoint &point);
```

(8) Inversement, proposez l'implémentation de :

```
WorldPoint toPhysical(const ScreenPoint &point);
```

🔧 Validations

Vérifiez votre implémentation en plaçant le joueur au centre du monde physique et en vérifiant qu'il apparaît bien au centre de l'écran.

Modifiez les coordonnées physiques du joueur dans votre code, et vérifiez que celui-ci s'affiche aux endroits attendus (bord droit, bord haut, etc.).

Pour ce TP, il est proposé d'utiliser la macro `_DEBUG` afin de pouvoir afficher des informations complémentaires lorsque le moteur est compilé en `Debug` :

(9) En mode `Debug`, affichez les axes x et y à l'aide de 2 traits rouges.

1.3 Déplacements du joueur : une première version simple

Le joueur est initialement positionné au centre du monde physique : ceci est codé en dur. Comme nous souhaitons pouvoir bouger le joueur, nous allons lui ajouter une position dans le monde physique et connecter cette position physique à l'affichage écran.

(1) Ajoutez 2 variables de type `double` afin de stocker la position x et y du joueur. Initialisez-les de manière à placer le joueur au centre du monde physique.

(2) Modifiez l'affichage du joueur à l'écran, en utilisant ses coordonnées physiques.

(3) Lorsque le joueur appuie sur la touche pour se déplacer vers la droite, augmentez la valeur de la position physique, par exemple de 1 mètre.

(4) Lorsque le joueur appuie sur la touche pour se déplacer vers la gauche, réduisez la valeur de la position physique, par exemple de 1 mètre.

(5) Faites de même pour les directions haut et bas.

🔧 Validations

Vérifiez que le joueur se déplace bien dans les directions voulues lors de l'appui des touches directionnelles.

Que se passe-t-il lorsque plusieurs touches directionnelles sont maintenues simultanément ? Le comportement obtenu vous paraît-il satisfaisant ?

1.4 Déplacements du joueur : une seconde version simple

Jusqu'à présent, le joueur ne se déplaçait qu'en réaction à un événement clavier. Cette approche est suffisante pour de nombreuses interfaces graphiques, mais elle montre rapidement ses limites dans un moteur de jeu : plusieurs touches simultanées sont difficiles à gérer, et le monde physique n'évolue qu'à l'arrivée d'un événement.

Pour contourner ce problème et obtenir un résultat plus fluide, nous ne pouvons pas nous baser sur les événements clavier : nous devons **régulièrement** – à chaque rafraîchissement de l'écran – interroger l'état du clavier. Le monde physique va donc évoluer à chaque mise à jour du moteur, et non plus à chaque événement clavier.

(1) Dans le code fourni initialement la fonction `update()` est appelée avec une vitesse de l'ordre de 50 FPS (*frames per second*). Voyez-vous dans le code où est appelée cette fonction ? et où est défini le taux de 50 FPS ?

(2) Dans la fonction `update()`, interrogez l'état du clavier pour déterminer si les touches directionnelles sont appuyées ou pas et modifiez la position physique en conséquence.

(3) Pensez à supprimer votre ancien code qui était déclenché lors des événements clavier. Le comportement obtenu vous paraît-il plus satisfaisant ? Essayez notamment d'appuyer simultanément sur plusieurs touches directionnelles.

2 Discrétisation – méthode d'Euler

🕒 de 3 à 6 h

Objectifs

- Voir les notions de dérivées discrètes.
- Comprendre la méthode d'Euler.
- Appliquer la méthode d'Euler à un système d'ordre 1.
- Étudier la stabilité de la méthode.

2.1 Dérivée discrète : vitesse du joueur

Lorsqu'un mobile parcourt une distance D durant un temps δt , sa **vitesse moyenne** $\bar{v}$ est définie par :

$$\bar{v} = \frac{D}{\delta t}$$

La vitesse moyenne est calculée entre deux instants éloignés l'un de l'autre : c'est une notion approximative. Avec cette information, on ne dispose pas d'informations sur le comportement de la vitesse entre les 2 instants.

Lorsque les 2 instants tendent à être de plus en plus proches, la vitesse moyenne se rapproche de plus en plus de la **vitesse instantanée**. Mathématiquement, la vitesse instantanée v est la limite de ce ratio lorsque la durée entre les 2 instants tend vers 0 :

$$v = \lim_{\delta t \rightarrow 0} \frac{D}{\delta t}$$

La limite du taux de variation d'une fonction est appelée nombre dérivé de la fonction. On parle alors d'une notion **continue**, car les deux instants utilisés dans le calcul peuvent être choisis aussi proches que l'on souhaite.

Lors d'une simulation, en général, on ne travaille pas avec un temps continu comme dans la réalité : le temps est découpé à l'aide d'un pas de temps et la simulation est réalisée uniquement sur ces pas de temps.

Exemple : si le moteur est paramétré à 50 FPS, alors le pas de temps est de $\frac{1}{50} = 0,02$ seconde. La position du joueur sera simulée aux temps 0 s ; 0,02 s ; 0,04 s ; etc. Entre chaque pas de temps, nous n'avons pas d'information précise.

💡 Dérivée discrète

La notion de **dérivée discrète** est la combinaison de ces éléments : il s'agit de la dérivée d'une fonction en utilisant des valeurs disponibles à des pas de temps différents, sans forcément qu'ils puissent être arbitrairement proches.

Par exemple, dans la figure 2, la courbe de la fonction P représente la position du joueur par rapport à une origine. Une approximation de la dérivée de P entre les points t_n et t_{n+1} peut s'écrire :

$$\frac{dP}{dt} \simeq \frac{P_{n+1} - P_n}{t_{n+1} - t_n}$$

Si l'on pouvait choisir t_n et t_{n+1} aussi proches que souhaité, alors la dérivée pourrait être calculée de manière exacte. Dans la réalité informatique, **un pas de temps a toujours une valeur finie, même très petite**.

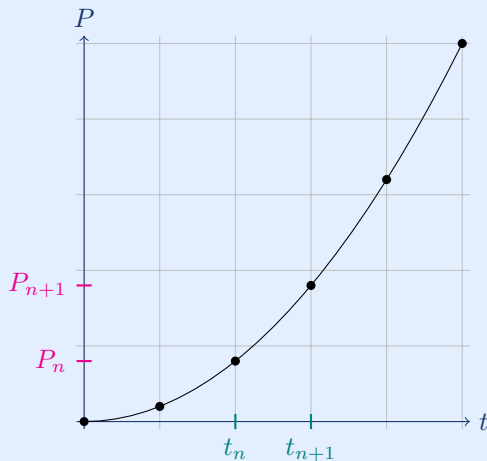


Figure 2. Position P du joueur, au cours du temps (courbe noire) et des points de simulation à différents pas de temps (points noirs).

(1) Pour la fonction affichée sur la figure 2, calculez une approximation discrète de la dérivée pour les points t_n , puis t_{n+1} . Vous pouvez considérer que les pas de grille valent une unité.

(2) On souhaite résoudre l'équation proposée ci-dessous. Déterminez la solution de cette équation (en considérant $P(0) = 0$), à l'aide de vos cours de maths des années précédentes.

$$\frac{dP}{dt} = 2$$

(3) Approximez cette équation en remplaçant la dérivée continue par la dérivée discrète, en utilisant un pas de temps Δt (vous pouvez prendre 1 seconde). Établir la relation de récurrence entre P_{n+1} et P_n : quel type de suite obtenez-vous ?

(4) Dans un tableur, tracez la solution théorique et quelques points avec la solution discrète. Comparez les résultats.

2.2 Équations différentielles d'ordre 1

Une **équation différentielle d'ordre 1** est une équation faisant intervenir une fonction et sa dérivée. Par exemple, l'équation suivante est une EDO d'ordre 1 :

$$\frac{df}{dt} = f$$

Sa solution est connue : $f(t) = A \times e^t$, où A est une constante et e^t la fonction exponentielle.

Remarque

On utilise souvent le raccourci EDO : équation différentielle ordinaire.

💡 Évolution de la vitesse du joueur

On considère un joueur de masse m , et accélérant de manière constante, grâce à un moteur. On note G sa propulsion.

Les forces de frottement avec l'air peuvent être modélisées à l'aide d'un coefficient k . Pour des vitesses suffisamment faibles, l'évolution de la vitesse v du joueur dans le temps est donnée par l'EDO d'ordre 1 :

$$m \frac{dv}{dt} = G - kv,$$

La solution de cette équation s'écrit :

$$v(t) = \frac{G}{k} + A \times e^{-\frac{t}{\tau}}$$

Avec A une constante, et $\tau = \frac{m}{k}$.

Remarque

La **propulsion** du joueur est une **accélération** : elle s'exprime en m.s^{-2} .

La vitesse s'exprime en m.s^{-1} . Avec ces informations, pouvez-vous en déduire les unités du coefficient k ?

(1) Montrez que la solution proposée ci-dessus est bien solution de l'équation différentielle proposée.

(2) Déterminez la constante A de la solution donnée.

(3) Tracez la solution dans un tableur pour une masse de 1208 kg et un coefficient de frottement de 1,8 USI. Vous pouvez prendre une vitesse initiale de 0 m.s^{-1} et une propulsion de 100 m.s^{-2} .

(4) Vers quelle vitesse se stabilise le joueur, d'après la forme de la solution ? de la courbe tracée ? cette vitesse d'équilibre vous semble-t-elle cohérente ?

(5) Rajoutez une seconde courbe avec un coefficient de frottement de 0,65 SI (joueur au profil aérodynamique plus performant). Comment la forme de la courbe est-elle modifiée ? (vitesse à l'équilibre, variation de la vitesse)

Remarque

SI ou USI signifie en **unités du Système International** : lorsque l'on ne souhaite pas écrire les unités de manière explicite (souvent par flemme).

(6) Le coefficient τ est appelé **temps caractéristique** : que se passe-t-il à ce temps pour les 2 cas étudiés ? pour la solution théorique ?

2.3 Méthode d'Euler explicite

La méthode d'Euler permet de passer du monde continu des mathématiques (avec ses solutions sous forme de fonctions), au monde discret des ordinateurs (avec des solutions estimées à certains instants uniquement).

Elle consiste à remplacer une équation différentielle par une relation de récurrence, facilement implémentable et résoluble à l'aide d'un ordinateur.

Calculer une solution approchée sur certains instants seulement nécessite de définir un **pas de temps** Δt : il s'agit du temps entre les instants que l'on va calculer. Exemple : si l'on prend un pas de temps d'une seconde, alors la solution approchée sera calculée pour les instants 0 s, 1 s, 2 s, etc.

Le principe à la base de la méthode d'Euler consiste à remplacer les dérivés par des approximations discrètes :

$$\frac{dv}{dt} \simeq \frac{v_{n+1} - v_n}{\Delta t},$$

où v_n est la vitesse approchée à l'itération n , et v_{n+1} est la vitesse approchée à l'itération suivante $n + 1$.

💡 Temps physique vs itérations

Si on considère que la simulation débute au temps 0 seconde, alors après 1 itération avec un pas de temps Δt , la simulation sera au temps physique Δt seconde(s).

De manière plus générale, après n itérations, le temps physique vaudra $n \times \Delta t$.

Note : cette relation est valable uniquement si le pas de temps est le même durant toute la simulation, ce qui sera choisi pour le TP. Dans certains cas, certains outils font varier le pas de temps.

Attention

Cette partie est une des plus importantes du TP : la méthode d'Euler introduit les principes fondamentaux des méthodes numériques utilisées en simulation.

(1) Que devient l'équation différentielle de l'évolution de la vitesse du joueur, une fois appliquée la méthode d'Euler ?

(2) Établir une relation de récurrence de la forme $v_{n+1} = a \times v_n + b$.

(3) Quel est le type de cette suite numérique ? Déterminez une expression de cette suite, sans relation de récurrence (*i.e.* exprimez v_n en fonction de m , k , G , Δt et v_0).

(4) Tracez dans le même tableur que précédemment, les points de la suite obtenue, avec un pas de temps de 200 secondes.

(5) Réduisez le pas de temps à une valeur de 10 secondes : que constatez-vous ?

(6) Augmentez le pas de temps à une valeur de 1000 secondes, puis 1500 secondes : que constatez-vous ?

(7) En mode **Debug**, ajouter l'affichage de la vitesse du joueur à l'écran, à l'aide de sa position actuelle et précédente.

2.4 Stabilité*

La méthode d'*Euler* explicite a consisté à remplacer la dérivée par une approximation discrète (membre de gauche). Le membre de droite a juste été modifié en remplaçant v par v_n : dans tout ce membre de droite, la vitesse est **évaluée à l'instant actuel** pour estimer la dérivée.

$$m \frac{v_{n+1} - v_n}{\Delta t} = G - k \times v_n$$

Ce choix a abouti à une suite arithmético-géométrique. L'expression de cette suite sous une forme sans récurrence a fait apparaître une expression de la forme $v_n = A \times r^n + B$.

(1) A quelle condition cette suite reste-t-elle bornée ? En déduire un critère sur le pas de temps à respecter pour éviter que la simulation ne diverge.

(2) A quelle condition cette suite ne présente pas d'oscillations de la vitesse ? En déduire un critère sur le pas de temps à respecter pour éviter que la simulation n'oscille.

2.5 Méthode d'*Euler* implicite*

La méthode d'*Euler* vue précédemment est qualifiée d'explicite car elle évalue le membre de droite à l'instant actuel (donc avec une vitesse $v \simeq v_n$ explicitement connue).

C'est un choix arbitraire : il aurait été tout à fait possible de choisir $v \simeq v_{n+1}$, c'est-à-dire à l'**instant ultérieur**, donc *a priori* pas encore connu (implicitement connu). La méthode d'*Euler* implicite s'écrit donc :

$$m \frac{v_{n+1} - v_n}{\Delta t} = G - k \times v_{n+1}$$

(1) Que devient la relation de récurrence pour le schéma implicite ?

(2) Affichez dans le tableur les vitesses simulées avec le schéma implicite et comparez avec les valeurs calculées à l'aide du schéma explicite.

(3) A partir de l'expression de la suite, déterminez un critère sur le pas de temps pour garantir la stabilité du schéma implicite (*i.e.* pas de divergence, pas d'oscillations).

(4) **Optionnel** : en récrivant la relation de récurrence sous forme d'une moyenne pondérée, voyez-vous un argument immédiat concernant la stabilité ?

Remarque

Ce résultat est relativement général : les schémas implicites sont très souvent stables alors que les schémas explicites sont souvent stables sous conditions.

3 Déplacements

de 2 à 3 h

Objectifs

- Appliquer la méthode d'*Euler* à un système d'ordre 2.
- Implémenter les déplacements du joueur en appliquant la méthode d'*Euler* au principe fondamental de la dynamique.
- Appliquer une force constante sur le joueur.

3.1 Relation fondamentale de la dynamique

La **relation fondamentale de la dynamique** – ou **loi de Newton** – nous dit que pour un objet de masse m , soumis à un ensemble de forces $\vec{F}$, celui-ci possède à une vitesse $\vec{v}$ telle que :

$$m \frac{d\vec{v}}{dt} = \vec{F}$$

Il est important de noter que cette loi est vectorielle. Ici nous cherchons à développer un moteur en deux dimensions, aussi, nous pouvons décomposer cette équation sous la forme de deux équations (une pour chaque axe) :

$$\begin{cases} m \times \frac{dv_x}{dt} = F_x \\ m \times \frac{dv_y}{dt} = F_y \end{cases}$$

où v_x et v_y sont les vitesses de l'objet dans les directions x et y ; et F_x et F_y sont les forces qui s'exercent sur l'objet dans les directions x et y .

Vous reconnaissez probablement deux EDO d'ordre 1, similaires à celle étudiée dans la partie précédente. Ici, nous avons rajouté une seconde dimension (nous sommes en 2D). Rajouter une dimension revient donc à rajouter une équation différentielle.

Il faut cependant noter que cela n'est pas plus complexe : nous avons deux EDO indépendantes l'une de l'autre. Lorsque nous appliquerons la méthode d'*Euler*, nous devrons l'implémenter deux fois : une fois dans la direction x , une autre fois dans la direction y . Le moteur devra donc appliquer la méthode pour chaque direction x et y .

Remarque

Vous remarquerez que le passage à un moteur 3D ne pose, d'un point de vue mathématique, pratiquement aucune difficulté supplémentaire : il suffit d'ajouter une troisième composante aux coordonnées (points, vecteurs) et une troisième équation différentielle.

3.2 Équation de Newton et *Euler* explicite

Vous voilà prêt·e à implémenter dans votre moteur, l'évolution du joueur en prenant en compte les lois de la physique.

(1) On note $v_{x,n}$ la vitesse du joueur à l'itération n , dans la direction x . À partir de la loi de Newton appliquée dans la direction des x , exprimez $v_{x,n+1}$ en fonction de m , $v_{x,n}$, F_x et Δt .

(2) Faites de même pour la vitesse selon la direction y .

(3) Dans votre code, ajoutez des variables (de type `double`) pour stocker les vitesses du joueur selon les directions x et y .

(4) Implémentez l'évolution de la vitesse du joueur à l'aide des relations de récurrence précédentes. Quelle valeur prendre pour Δt ? Pour les forces F_x et F_y , vous pouvez, dans un premier temps, les définir à l'aide de fonctions de la forme :

```
double F_x() const
{
    return 0.0;
}
```

(5) Implémentez une force verticale constante, dirigée vers le bas (par exemple avec une intensité de -10 USI, comme la gravité sur Terre). Comment varie la vitesse du joueur dans le temps?

(6) On souhaite déterminer l'évolution de la position du joueur (et pas uniquement sa vitesse). On rappelle que la vitesse est la dérivée de la position par rapport au temps :

$$v_x = \frac{dx}{dt} \text{ et } v_y = \frac{dy}{dt}$$

Si l'on note x_n la position du joueur, selon l'axe x , à l'itération n , appliquez la méthode d'*Euler* à la première relation pour déterminer la relation entre x_{n+1} et x_n .

(7) Faites de même pour y_{n+1} en fonction de y_n .

(8) Dans votre code, ajoutez des variables (de type `double`) pour stocker les positions du joueur selon les directions x et y .

(9) Implémentez l'évolution de la position du joueur à l'aide des relations de récurrence précédentes.

(10) Comment évolue la position du joueur dans le temps avec une gravité constante dirigée vers le bas? Le mouvement du joueur sur l'écran est-il cohérent avec ce qui est attendu?

💡 Équations différentielles d'ordre 2

Une équation différentielle est dite d'ordre 2 lorsqu'elle fait intervenir la dérivée seconde de la fonction recherchée. Par exemple, la loi de Newton peut s'écrire sous la forme :

$$m \frac{d^2x}{dt^2} = F_x$$

où le terme $\frac{d^2x}{dt^2}$ représente la dérivée de la dérivée de la fonction $x(t)$.

Ici, nous avons écrit cette loi sous la forme :

$$\begin{cases} m \times \frac{dv_x}{dt} = F_x \\ v_x = \frac{dx}{dt} \end{cases}$$

Nous avons appliqué la méthode d'*Euler* à la première équation, pour déterminer la vitesse. Et nous avons ensuite appliqué la méthode d'*Euler* une nouvelle fois à la seconde équation, pour déterminer la position.

Cette transformation est très utilisée en simulation numérique. Les solveurs d'équations différentielles commencent généralement par réécrire les équations d'ordre supérieur sous la forme d'un système d'équations du premier ordre.

Nous avons transformé une EDO d'ordre 2 en deux EDO d'ordre 1. Aussi, si vous savez calculer une solution d'une EDO d'ordre 1 à l'aide de la méthode d'*Euler*, vous savez résoudre une EDO d'ordre quelconque, avec un nombre quelconque de dimensions (en 2D, en 3D, etc.).

3.3 Méthode d'Euler d'ordre 2*

Précédemment, la méthode d'Euler a été appliquée à une équation différentielle d'ordre 1. La relation fondamentale de la dynamique est une équation différentielle d'ordre 2. Nous l'avons transformée en système de 2 équations différentielles d'ordre 1, afin de pouvoir appliquer la méthode d'Euler.

Nous allons voir ici comment discrétiser directement une équation différentielle d'ordre 2.

Pour rappel, l'approximation d'une dérivée à l'aide de la dérivée discrète consiste à remplacer

$$\frac{df}{dt} \simeq \frac{F_{n+1} - F_n}{\Delta t}$$

où f est une fonction (continue et dérivable) et F l'équivalent discret de cette fonction.

Pour les questions suivantes, on se référera à la figure 3 concernant les notations.

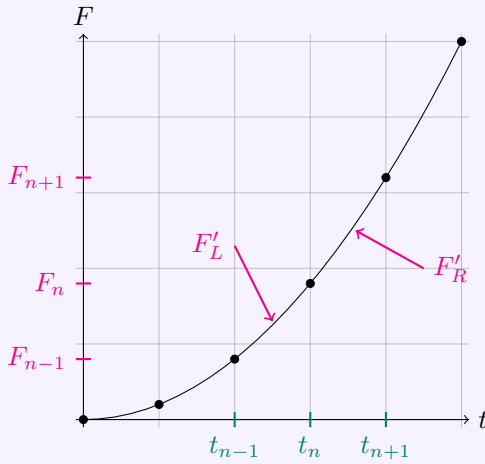


Figure 3. Fonction f (en noir) et équivalent discret F (points noirs).

On considère ici 3 temps : t_{n-1} , t_n et t_{n+1} .

(1) Que vaut la dérivée discrète de la fonction f , selon t , entre les points n et $n+1$? On notera F'_R cette valeur.

(2) Que vaut la dérivée discrète de la fonction f , selon t , entre les points $n-1$ et n ? On notera F'_L cette valeur.

(3) La dérivée seconde étant la dérivée de la dérivée, on peut écrire :

$$\frac{d^2 f}{dt^2} = \frac{df'}{dt} \simeq \frac{F'_R - F'_L}{\Delta t}$$

En remplaçant les dérivées de f par les dérivées discrètes déterminées juste avant, en déduire que (c'est le schéma centré d'ordre 2) :

$$\frac{d^2 f}{dt^2} \simeq \frac{F_{n+1} - 2F_n + F_{n-1}}{\Delta t^2}$$

(4) L'accélération a_x selon l'axe des x étant la dérivée seconde de la position $x(t)$:

$$a_x = \frac{d^2 x}{dt^2}$$

écrire le schéma d'ordre 2 selon l'axe des x de l'équation de Newton, pour des forces F_x .

(5) En écrivant les 2 relations de récurrence d'ordre 1, et en les combinant, déterminez la relation de récurrence pour x_{n+1} en fonction de x_n et x_{n-1} .

(6) Retrouvez-vous la même relation qu'en utilisant le schéma centré d'ordre 2 ?

(7) Lors de l'implémentation informatique des 2 relations d'ordre 1, devez-vous calculer en premier la vitesse ou la position, pour retrouver le schéma centré d'ordre 2 ?

4 Forces

🕒 de 2 à 4 h

Objectifs

- Ajouter la propulsion du joueur.
- Appliquer des forces de frottements sur le joueur.

4.1 Propulsion du joueur

Dans cette section, vous pouvez enrichir votre moteur avec une ou plusieurs forces. Ajouter une force suffit à l'enrichir, la méthode d'*Euler* ne changera pas : la base du moteur est faite.

(1) Modifiez vos fonctions F_x et F_y afin d'ajouter comme force la propulsion du joueur dans les quatre directions, selon s'il appuie ou non sur les touches directionnelles. Vous pouvez choisir une propulsion constante de l'ordre de 100 USI.

(2) Validez le mouvement de votre joueur en testant l'implémentation.

(3) Comment feriez-vous pour cumuler propulsion du joueur et gravité dirigée vers le bas ?

4.2 Forces de frottement

On propose d'ajouter une force de frottements visqueux sur le joueur. Une telle force est un peu plus complexe à implémenter. En effet :

1. Le frottement est un vecteur 2D parallèle à la vitesse du joueur.
2. Le frottement est dirigé dans le sens inverse de la vitesse (sinon il jouerait le rôle d'une propulsion).
3. La norme du vecteur frottement est de la forme $k \times v$ ou $k \times v^2$, où k est un coefficient de frottement et v la vitesse du joueur.

(1) A chaque itération, calculez la vitesse du joueur (*i.e.* la norme du vecteur vitesse).

(2) En déduire la norme du vecteur frottements.

(3) Déterminez le vecteur frottement, connaissant sa norme et le vecteur vitesse du joueur.

(4) Déterminez les coordonnées du vecteur frottement selon les axes x et y . Utilisez ces valeurs comme force complémentaire.

(5) Testez votre implémentation : le joueur est-il bien ralenti par les frottements ? possède-t-il une vitesse limite ?

Remarque

Lorsque les vitesses sont faibles, les frottements sont bien modélisés à l'aide d'une force en $k \times v$.

En revanche, pour des vitesses élevées, il est standard de choisir un frottement de la forme $k \times v^2$, où la vitesse intervient au carré.

4.3 Forces complexes*

Les forces implémentées précédemment sont relativement simples. Des forces plus complexes peuvent être ajoutées comme par exemple les forces centrales.

Une force est dite centrale si elle est toujours dirigée vers un centre. C'est le cas de la gravité lorsque l'on est dans l'espace : le Soleil exerce par exemple sur la Terre une force dirigée selon le vecteur Terre $\rightarrow$ Soleil, donc toujours dirigée vers le Soleil. Si l'on note $\vec{r}$ le vecteur Terre $\rightarrow$ Soleil de norme 1, la force $\vec{g}$ de l'attraction du Soleil sur la Terre s'écrit :

$$\vec{g} = G \frac{m \times M}{r^2} \vec{r},$$

où m est la masse de la Terre, M la masse du Soleil, G la constante gravitationnelle et r la distance entre la Terre et le Soleil.

- (1) Implémentez une force centrale dans votre moteur.
- (2) Comment se comporte votre moteur lorsque le pas de temps varie ?
- (3) Vous pouvez également choisir d'implémenter une force centrale de type attraction – répulsion électrique : la force est de même forme que l'attraction due à la gravitation exceptée que les masses sont remplacées par des charges – positives ou négatives.

5 Géométrie du joueur

de 3 à 6 h

Objectifs

- Faire tourner le joueur dans la direction du mouvement.
- Intégrer les rebonds sur les bords de l'écran.

5.1 Rotation du joueur

Dans cette section, on se propose de tourner le joueur dans la direction du mouvement, pour plus de réalisme.

- (1) Si l'affichage de votre moteur est uniquement un point à l'écran, modifiez votre joueur à l'aide d'une petite image (par exemple une voiture).
- (2) Déplacez votre joueur à l'écran à l'aide du clavier : sans rotation, le joueur sera toujours dirigé dans la même direction.
- (3) On souhaite aligner la direction du joueur (par exemple la direction de la voiture), dans la même direction que le déplacement. Comment feriez-vous ?
- (4) Faites-le.

5.2 Rebonds sur les bords de l'écran

On propose comme évolution du moteur que le joueur ne puisse pas sortir de l'écran.

Important : la phrase précédente est relativement ambiguë ! En effet, nous ne devons pas imposer des limites dans le monde écran à notre moteur, car celui ne connaît que les coordonnées physiques.

- (1) Déterminez les coordonnées écran minimale et maximale que vous souhaitez imposer à votre joueur.
- (2) En déduire les coordonnées physiques équivalentes.
- (3) Implémentez une première version simple de limites : si le joueur sort du cadre prévu pour sa nouvelle position, n'appliquez pas cette position, mais conservez l'ancienne.
- (4) Testez cette implémentation : le joueur reste-t-il borné au cadre prévu ? si vous accélérez le joueur vers le haut, pourquoi reste-t-il coincé quelques instants en haut de l'écran ?
- (5) Comment corrigez cet effet de bord ?
- (6) Améliorez le cadre en proposant un rebond simple sur les bords : en cas de sortie du cadre, la vitesse du joueur doit changer de signe (dans la direction perpendiculaire au bord), afin de simuler un rebond.
- (7) Testez l'implémentation : voyez-vous des effets de bord pour cette implémentation ?

5.3 Rebonds précis* et/ou conservatifs***

Les rebonds implémentés précédemment sont relativement simples, mais des effets de bords peuvent apparaître.

(1) Entre l'itération n et $n + 1$, le joueur peut franchir partiellement ou totalement un bord. Dans ce cas, sa vitesse dans cette direction est inversée et il s'en éloignera ... mais en ayant quand même légèrement franchi la limite. Calculez la distance pouvant être parcourue avant le contact strict : déplacez le mobile jusqu'au contact, et utilisez la vitesse inversée pour le reste du pas de temps.

(2) Cela améliore-t-il les rebonds ?

(3) Supprimez temporairement toutes les forces de frottement. A chaque itération calculez l'énergie totale du joueur :

$$E(t) = \frac{1}{2}mv^2 + mgy$$

où g est la valeur de la gravité choisie.

Cette quantité est normalement constante dans le temps s'il n'y a pas de frottements ni de propulsion. Est-ce le cas ? est-ce le cas lors d'un rebond ?

(4) ☠ Pourquoy un rebond n'est pas conservatif (*i.e.* l'énergie totale n'est pas conservée) ?

(5) ☠ ☠ ☠ Proposez une modification de l'algorithme permettant de mieux conserver l'énergie totale lors des rebonds.

(6) Quels intérêts à vouloir rendre les rebonds plus précis ? conservatifs ?

6 Autres entités

🕒 de 6 à 8 h

Objectifs

- Ajout d'autres entités au moteur de jeu (ennemis, ressources, autres joueurs, etc.).
- Intégrer les collisions avec les autres entités et émettre des signaux.
- Préparer une interface pour gérer les déplacements des autres entités.

6.1 Ajout d'autres entités

Dans une première version du moteur, un seul joueur est simulé. Cette section est davantage consacrée à l'architecture du moteur qu'à la simulation physique. Ajoutez plusieurs entités, en prenant soin de distinguer celles qui sont contrôlables avec le clavier, celles qui sont fixes, etc.

(1) Créez une entité abstraite, représentant une entité pouvant être intégrée dans le moteur de jeu.

(2) Réimplémentez le joueur principal. Pensez à rendre paramétrables les touches de direction, afin de pouvoir prendre en charge 2 joueurs ou plus.

(3) Implémentez des entités fixes, ou avec un léger mouvement prédéfini (par exemple faisant des aller-retours).

6.2 Collisions

Dans cette section, on s'intéresse aux collisions entre entités. Gérer les collisions peut intervenir dans plusieurs cas : collecte des ressources participant au score, attaque par un ennemi conduisant au *game-over*, etc.

(1) Chaque entité est représentée par son centre et son rayon moyen. Implémentez la détection de l'intersection entre deux cercles de rayons différents r_1 et r_2 .

(2) A chaque mise à jour du moteur, détectez l'intersection entre le joueur et une autre entité. Notifiez le contact entre le joueur et une entité (dans le cas où vous utilisez une bibliothèque comme `Qt`, vous pouvez émettre un `signal`).

(3) Lorsque le joueur entre en contact avec une ressource à récolter, augmentez son score et supprimez l'entité.

(4) Lorsque le joueur entre en contact avec un ennemi, quittez le jeu ou bien diminuez sa barre de vie.

6.3 Collisions élastiques*

Certaines entités peuvent être neutres pour le joueur (ni ennemi, ni à récolter). Dans ce cas, une collision élastique peut être implémentée. Des collisions élastiques peuvent également être implémentées avec des segments de droite

- (1) Faites le schéma d'un rebond d'un cercle sur un segment de droite.
- (2) Déterminez le moment de la collision, et implémentez le rebond élastique. On pourra considérer que seule la composante de la vitesse perpendiculaire au segment change de signe.
- (3) 🧠 Si vous souhaitez que votre moteur le permette, modifiez les rebonds sur les segments de droite de telle manière qu'ils agissent uniquement dans un sens (dans l'autre sens, le segment peut être traversé).
- (4) 🧠 🧠 🧠 Si vous souhaitez que votre moteur le permette, implémentez les rebonds élastiques entre deux entités (supposées être deux cercles de masses et rayons différents). Pour cette partie, il est fortement conseillé de chercher sur internet ou de demander à une IA les formules à appliquer, tout en gardant un esprit critique sur ses propositions (est-ce que cela fonctionne ? dans tous les cas ? est-ce réaliste ?).

6.4 Ennemi suivant le joueur

On souhaite rajouter des entités ennemies qui vont chercher à suivre le joueur afin de provoquer un *game-over* (par exemple) lors du contact.

(1) Comment feriez-vous pour implémenter un suivi, basé sur des règles ? Testez votre idée. En tant qu'entité ennemie, vous avez le droit de savoir où se trouve la cible à atteindre. Vous n'avez en revanche pas le droit de modifier votre position, ni votre vitesse. Vous ne pouvez agir que sur vos propulsions (comme un joueur avec le pavé directionnel).

(2) Si vous n'avez pas d'idée, vous pouvez utiliser des propulsions proportionnelles aux distances séparant l'ennemi de sa cible, selon chacun des axes.

(3) Ce type de suivi est-il efficace ? et pourquoi ?

(4) Utiliser la distance restant à parcourir comme propulsion peut conduire à un ennemi beaucoup trop nerveux et rapide ... et inefficace. Il est nécessaire de ne pas accélérer trop proche de la cible, surtout avec une vitesse élevée. Comment modifieriez-vous votre code pour prendre cela en compte ?

(5) Cette implémentation est-elle plus satisfaisante ?

(6) Comment change le comportement de votre ennemi si vous faites varier :

1. Son coefficient de frottement ?
2. Sa masse ?
3. La part de la distance à la cible pour sa propulsion ?
4. La part de la différence de vitesse avec la cible pour sa propulsion ?

(7) Proposez plusieurs profils d'ennemis (facile, intermédiaire, difficile, rapide, lourd, prudent, etc.) en jouant uniquement sur les paramètres précédents.

6.5 Comment tu me suis?*

On a récupéré un code qui définit les propulsions x et y d'une entité suiveuse. Dans ce code, le joueur est de masse 1 et de coefficient de frottement 0,1 (pour une loi en v^2).

L'entité suiveuse a une masse de 10 pour un coefficient de frottement de 3. Cette entité suit le joueur avec les propulsions suivantes :

```
double propulsion_x() const
{
    if (!_target)
        return 0.0;

    double dx = _target->_x - _x;
    double dy = _target->_y - _y;

    double d2 = dx * dx + dy * dy;
    double d = sqrt(d2);

    return (d2 - 625/d2) * (dx / d) + 100 * dy / d;
}

double propulsion_y() const
{
    if (!_target)
        return 0.0;

    double dx = _target->_x - _x;
    double dy = _target->_y - _y;

    double d2 = dx * dx + dy * dy;
    double d = sqrt(d2);

    return (d2 - 625/d2) * (dy / d) - 100 * dx / d;
}
```

- (1) Implémentez ce code et testez son comportement.
- (2) Comment se comporte l'entité suiveuse? est-elle rapide ou lente? part-elle loin ou pas? touche-t-elle la cible ou pas?
- (3) Dans un premier temps, on s'intéresse au premier terme de la propulsion : désactivez le second terme (par exemple en remplaçant 100 par 0). Faites un schéma avec la cible, et pour différentes positions de l'entité suiveuse, déterminez le vecteur propulsion.
- (4) Vers quoi le vecteur propulsion est-il toujours dirigé? est-ce cohérent avec le comportement de l'entité?
- (5) Y a-t-il un équilibre possible? si oui, pouvez-vous décrire plus en détail cet état d'équilibre?
- (6) Rajoutez le second terme de la propulsion. Si vous reprenez votre schéma, comment sont modifiées les vecteurs propulsion?
- (7) Dans quelle direction pointe ce vecteur supplémentaire? quel effet apporte-t-il?
- (8) Comment décririez-vous le comportement de cette entité suiveuse vis-à-vis de sa cible? à quoi cela peut-il vous servir dans un moteur de jeu?

6.6 Ennemi avec IA simple***

Dans cette section, on propose d'entraîner un ennemi afin qu'il se comporte de manière similaire à notre manière de jouer.

Un exemple simplifié est proposé ici pour comprendre la méthode, avant de l'appliquer à notre cas un peu plus complexe. La figure 4 montre un ensemble de points : il s'agit de mesures réalisées pendant que l'on entraîne notre ennemi.

L'axe horizontal correspond à la distance d que l'on constate entre la cible et l'entité. L'axe vertical correspond à la propulsion p appliquée par le joueur dans ce cas-là.

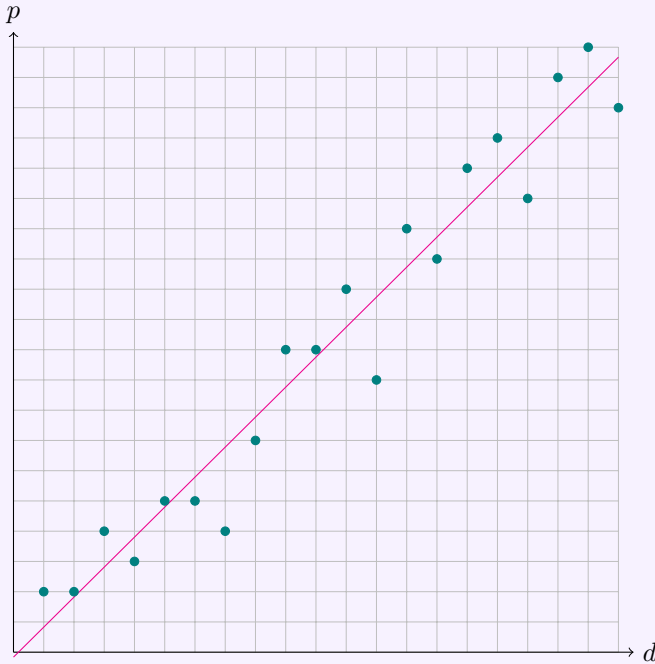


Figure 4. Exemple de jeu de données (points verts – propulsion en fonction de la distance). La droite de régression linéaire est affichée en rouge.

La phase d'entraînement va produire un très grand nombre de points : avec un taux de rafraîchissement de 50 FPS, un entraînement de 2 minutes conduit à environ $2 \times 60 \times 50 = 6000$ points.

On pourrait être tenté de garder tous les points, ou quelques-uns : durant le jeu, notre ennemi irait prendre le point le plus proche lorsqu'il est à une distance d de la cible et appliquerait la propulsion associée.

Cette méthode est lente et surtout elle ne généralise pas le comportement moyen du joueur ayant entraîné l'ennemi. L'idée est de modéliser cet ensemble de points par une fonction $p(d)$ et de choisir une forme de courbe simple.

La figure 4 montre la droite de régression linéaire, en rouge. C'est une fonction très simple puisqu'il suffit de connaître 2 coefficients a et b pour décrire tous les points :

$$p(d) \simeq a \times d + b$$

Faire une régression sur un ensemble de données, c'est choisir volontairement de modéliser toutes les données à l'aide d'une fonction simple : on y gagne en rapidité, en généralisation ... mais on y perd en précision.

Déterminer les coefficients a et b de l'équation de la droite qui passe au mieux vers tous les points est un résultat connu (N est le nombre de points) :

$$a = \frac{N \sum_{i=1}^N d_i p_i - \sum_{i=1}^N d_i \sum_{i=1}^N p_i}{N \sum_{i=1}^N d_i^2 - \left(\sum_{i=1}^N d_i \right)^2},$$

$$b = \frac{1}{N} \left(\sum_{i=1}^N p_i - a \sum_{i=1}^N d_i \right).$$

Note : pour celles et ceux intéressés sur comment on aboutit à ces résultats, un document annexe est disponible sur Moodle.

Dans ce que nous venons de voir, la distance d et la propulsion p concernaient une entité dans un monde à une dimension. Complexifions un peu le modèle : passons dans un monde à deux dimensions. Nous traiterons donc avec les quantités suivantes :

- La distance entre l'entité et le joueur, selon l'axe x , notée d_x .
- La distance entre l'entité et le joueur, selon l'axe y , notée d_y .
- La propulsion de l'entité selon x , notée p_x .
- La propulsion de l'entité selon y , notée p_y .

Concentrons-nous d'abord sur la propulsion selon x . Durant la phase d'entraînement, des points de mesure sont enregistrés. Ils sont de la forme :

$$(d_x, d_y) \rightarrow p_x(d_x, d_y)$$

Autrement dit, la propulsion selon x est une **fonction de 2 variables** : d_x et d_y . La figure 5 montre un ensemble de points enregistrés durant l'entraînement : pour une distance d_x et d_y de la cible, on a enregistré la propulsion selon x .

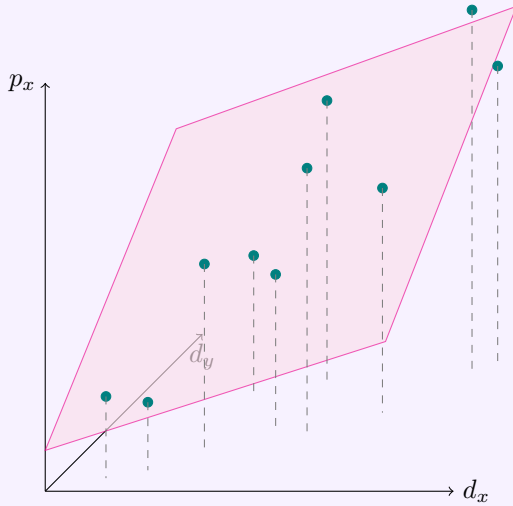


Figure 5. Exemple de jeu de données (points verts – propulsion selon x en fonction de la distance selon x et selon y). Le plan de régression est affiché en rouge.

Le plan représentant au mieux les données possède une équation de la forme :

$$p_x(d_x, d_y) = a + b \times d_x + c \times d_y$$

Pour la propulsion selon l'axe des y , on pourrait procéder de la même manière et déterminer l'équation d'un second plan :

$$p_y(d_x, d_y) = a' + b' \times d_x + c' \times d_y$$

Généralisons encore : nous choisissons ici d'entraîner le modèle avec encore plus de données. Nous allons enregistrer à chaque pas de temps :

- La distance entre la cible et l'entité selon x : d_x .
- La distance entre la cible et l'entité selon y : d_y .
- La vitesse de la cible selon x : $v_{t,x}$.
- La vitesse de la cible selon y : $v_{t,y}$.
- La vitesse de l'entité selon x : v_x .
- La vitesse de l'entité selon y : v_y .

Et nous allons chercher les équations des hyperplans :

$$p_x(d_x, d_y, v_{t,x}, v_{t,y}, v_x, v_y) = c_0 + c_1 d_x + c_2 d_y + c_3 v_{t,x} + c_4 v_{t,y} + c_5 v_x + c_6 v_y$$

$$p_y(d_x, d_y, v_{t,x}, v_{t,y}, v_x, v_y) = c'_0 + c'_1 d_x + c'_2 d_y + c'_3 v_{t,x} + c'_4 v_{t,y} + c'_5 v_x + c'_6 v_y$$

Les coefficients c_i et c'_i sont déterminés de manière à ce que les hyperplans passent au mieux par les données mesurées. Ils sont donnés par la méthode des moindres carrés.

Si les données d'entraînement changent, les coefficients changent également : le comportement de l'ennemi dépend donc directement des exemples qui lui ont été fournis.

Dans le cadre de ce TP, la fonction `computeHyperplanCoefficients()` permet de les déterminer sans entrer dans le détail des calculs (grâce à la bibliothèque **Eigen**).

Important. Ici, le comportement de l'ennemi est modélisé par un hyperplan : c'est l'un des modèles les plus simples utilisés en apprentissage automatique. Des modèles plus élaborés (comme par exemple les **réseaux de neurones**) reposent sur le même principe général : apprendre une fonction à partir d'exemples.

(1) Implémentez un joueur à trajectoire enregistrée : une implémentation qui sauvegarde les positions à chaque pas de temps ; puis une implémentation qui permet de rejouer la trajectoire.

(2) Implémentez une entité d'entraînement : c'est cette entité que vous allez contrôler, tout en rejouant la trajectoire enregistrée du joueur, en essayant de suivre et de toucher au mieux le joueur.

(3) Enregistrez les données de l'entité d'entraînement : position relative de la cible, vitesse de la cible, vitesse de l'entité, et propulsions x et y que vous appliquez lorsque vous jouez.

(4) A partir de ces données d'entraînement, déterminez les coefficients des hyperplans pour les propulsions x et y .

(5) Implémentez une entité de type ennemi IA qui utilise les propulsions x et y déterminées par les équations des hyperplans.

(6) Votre ennemi IA arrive-t-il à suivre le joueur ?

(7) Testez plusieurs phases d'entraînement en modifiant **volontairement** votre manière de jouer de manière à essayer de produire des ennemis IA avec des niveaux de difficulté croissants.

Remarque

Un **hyperplan** est la généralisation de la notion de plan pour des dimensions supérieures à 2.

7 Pour aller plus loin

7.1 Fond d'écran fixe

Pour terminer ce moteur de jeu, on propose l'ajout d'un fond d'écran à la scène.

(1) Ajouter un fond d'écran fixe à votre scène.

(2) Paramétrez les éléments de votre moteur (bords, segments avec collision, etc.) afin que ceux-ci soient cohérents avec votre fond. Par exemple : paramétrez des segments bloquants le long d'une piste de course.

(3) 🐼 Animez très légèrement votre fond en ajoutant des entités sans interactions avec le joueur.

(4) 🐼🐼 Animez directement le fond d'écran en appliquant des filtres en temps réel sur les pixels de l'image.

7.2 Scènes

Votre moteur actuel gère une seule scène : un fond d'écran, un ou plusieurs joueurs, des ressources ramassées, une mort possible. Comment mettriez-vous en place une gestion des scènes dans le moteur (transitions, *game-over*, etc.) ?

7.3 Fond procédural simulé***

Attention : cette section est volontairement plus complexe. Elle ne présente pas d'intérêt majeur pour votre moteur. Son seul intérêt : vous présenter les **dérivées partielles**. Ces notions sont importantes et utiles pour celles et ceux qui souhaitent poursuivre leurs études au-delà du BUT.

💡 Dérivée partielle

La dérivée partielle généralise la notion de dérivée aux fonctions de **plusieurs variables**.

La **dérivée partielle** de la fonction f , par rapport à la variable x est la dérivée de la fonction f par rapport à la variable x , en maintenant toutes les autres variables constantes. Elle est notée (et se lit « d rond f sur d rond x »)

$$\frac{\partial f}{\partial x}$$

Exemple :

$$\frac{\partial}{\partial x} (6xy + x^2 + y^2) = 6y + 2x$$

Nous avons vu dans ce TP les équations différentielles (ordinaires) : ce sont des équations dont la solution est une fonction, et qui font intervenir la fonction et ses dérivées. Une EDO s'applique aux fonctions d'**une seule variable**.

Lorsque la fonction recherchée dépend de plusieurs variables et est solution d'une équation faisant intervenir ses dérivées partielles, on parle d'**équation aux dérivées partielles** (EDP).

Par exemple, dans le cas de la conduction de la chaleur, en une dimension, la température T , qui dépend à la fois du temps t et la position x vérifie l'EDP :

$$\rho C_p \frac{\partial T}{\partial t} = \lambda \frac{\partial^2 T}{\partial x^2}$$

Résoudre numériquement une telle équation est toujours possible avec la méthode d'*Euler* : il suffit d'appliquer la méthode à la variable temporelle t et à la variable d'espace x . Par exemple, si $T_i^{(n)}$ représente la température à la position i , et à l'itération temporelle n , alors cette équation peut être discrétisée ainsi :

$$\rho C_p \frac{T_i^{(n+1)} - T_i^{(n)}}{\Delta t} = \lambda \frac{T_{i-1}^{(n)} - 2T_i^{(n)} + T_{i+1}^{(n)}}{\Delta x^2}$$

Si vous souhaitez vous entraîner avec ces concepts, vous pouvez implémenter (avec la méthode d'*Euler* explicite) le système différentiel non linéaire suivant – modèle de **FitzHugh-Nagumo** modifié (🧠🧠🧠) :

$$\begin{cases} \frac{\partial u}{\partial t} = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + u - \frac{u^3}{3} - v \\ \frac{\partial v}{\partial t} = \varepsilon(u - \beta v) \end{cases}$$

On pourra utiliser les paramètres suivants : $\alpha = 2$, $\varepsilon = 1$ et $\beta = 1$. Pensez à initialiser les champs u et v à des valeurs non nulles.

Le champ u peut être utilisé pour dessiner l'arrière plan. Une coloration de la forme ci-dessous peut par exemple être testée :

```
double angle = (u + 1.0) * std::numbers::pi;

uint8_t r = (uint8_t)((std::sin(angle * 2.0) * 0.5 + 0.5) * 255);
uint8_t g = (uint8_t)((std::sin(angle * 3.0 + 2.0) * 0.5 + 0.5) * 255);
uint8_t b = (uint8_t)((std::sin(angle * 4.0 + 4.0) * 0.5 + 0.5) * 255);
```

7.4 Adapter un moteur existant***

Vous avez développé dans ce TP un petit moteur de jeu intégrant des déplacements pilotés par les lois de la physique. Cette architecture n'est pas universelle : certains moteurs existants proposent d'autres choix de conception.

Par exemple, le projet [cangine](#) est un petit moteur de jeu développé en `c++` et `OpenGL`.

(1) L'architecture de ce moteur permet-elle naturellement d'intégrer des déplacements pilotés par des forces physiques ? Pourquoi ?

(2) Après avoir étudié le code de ce moteur, comment feriez-vous pour intégrer les développements que vous avez faits dans ce TP ? Quels composants, classes ou systèmes modifieriez-vous ?

(3) Proposez une implémentation suffisamment générique pour rester cohérente avec l'architecture du moteur (sans régressions, ni ralentissements, ni obligation d'utiliser votre implémentation).

(4) Si votre implémentation vous paraît suffisamment propre et compatible avec les objectifs du projet, vous pouvez proposer une **Pull Request** au développeur.

L'objectif n'est pas uniquement de faire fonctionner votre modification, mais également de respecter les choix d'architecture du projet auquel vous contribuez.

Note de fin

L'objectif de ce TP était d'aborder quelques notions mathématiques utiles au développement d'un petit moteur de jeu pédagogique.

Il ne prétend pas à l'exhaustivité ! Mais plus à montrer que le but d'un moteur est de poser des bases ; et que construire plus avec ces bases n'est pas le plus difficile.

Par exemple, serait-ce maintenant difficile :

- D'ajouter de nouvelles forces (aimants, trous noirs, ...) ? **probablement non**
- De passer à un moteur 3D ? **probablement non**
- De rajouter des pertes lors des rebonds ? **probablement non**
- D'augmenter le nombre de scènes, de joueurs, d'ennemis ? **probablement non**
- De rajouter des échanges via le réseau pour jouer à distance ? **probablement non**
- D'afficher de la fumée simulée sur carte graphique ? là, oui ... un TP entier aurait été nécessaire !